

Contents

(CDI System) 1

. 1

: 1

: 1

. 1

1: (Network Security) 1

2: (Application Security) 2

3: (Data Security) 4

4: (Monitoring & Auditing) 5

. 6

1. 6

2. 7

3. (Session Policy) 7

. 7

. 7

. 8

HIPAA Compliance Checklist 8

. 8

Penetration Testing 8

. 9

(CDI System)

:
: 2025
: -

:
CDI .
:
HIPAA (Health Insurance Portability and Accountability Act) ISO/IEC 27001 () NIST Cybersecurity Framework
OWASP Top 10 ()

1: (Network Security)

1.1

TLS 1.3
HTTPS (HTTP)
Certificate Pinning
Perfect Forward Secrecy (PFS)

Configuration:

- TLS Version: 1.3
- Cipher Suites: ECDHE-RSA-AES256-GCM-SHA384
- Certificate: Let's Encrypt ()

1.2 (Firewall)

Rules:

- Allow: 443 (HTTPS)
- Allow: 80 (HTTP redirect to 443)
- Block: All other ports
- Rate Limiting: 100 requests/minute per IP
- DDoS Protection: Cloudflare/AWS Shield

2: (Application Security)

2.1 (Authentication) JWT (JSON Web Tokens):

```
# Token
{
  "algorithm": "HS256",
  "token_lifetime": "24 hours",
  "secret_key": "256-bit random key",
  "payload": {
    "user_id": "uuid",
    "role": "admin|supervisor|user",
    "exp": "expiration_timestamp"
  }
}
```

- ```
#
- Signature verification
- Expiration check
- Token blacklist
```

### MFA (Multi-Factor Authentication):

Flow:

1. Username + Password (Something you know)
2. OTP via Email (Something you have)

OTP Specs:

- Length: 6 digits
- Validity: 30 minutes
- Storage: Encrypted in MongoDB
- Rate Limit: 5 attempts per hour
- Cooldown: 1 minute between resends

### 2.2 (Authorization) \*\* RBAC (Role-Based Access Control):\*\*

```
const roles = {
 admin: {
 permissions: [
 'user:create', 'user:read', 'user:update', 'user:delete',
 'note:*', 'analysis:*', 'system:*', 'audit:read'
]
 },
 supervisor: {
 permissions: [
 'user:read', 'user:update',
```

```

 'note:read', 'analysis:read',
 'team:manage', 'reports:read'
]
},
user: {
 permissions: [
 'note:create', 'note:read', 'note:update', 'note:delete',
 'analysis:read', 'chat:use'
]
}
}
}

```

### 2.3 SQL Injection Protection:

```

MongoDB (NoSQL)
Parameterized queries
SQL

```

Example:

```

db.users.find(f"email='{email}'") # Unsafe
db.users.find({"email": email}) # Safe

```

### XSS (Cross-Site Scripting) Protection:

```

// Frontend Sanitization
import DOMPurify from 'dompurify';

const clean = DOMPurify.sanitize(userInput);

// Content Security Policy (CSP)
{
 "Content-Security-Policy":
 "default-src 'self'; " +
 "script-src 'self' 'unsafe-inline'; " +
 "style-src 'self' 'unsafe-inline'; " +
 "img-src 'self' data: https;";
}

```

### CSRF Protection:

```

CORS Configuration
CORS(app,
 origins=["https://cdi-system.sa"],
 credentials=True,
 methods=["GET", "POST", "PUT", "DELETE"],
 allow_headers=["Content-Type", "Authorization"]
)

```

```

CSRF Token Forms

```

### Rate Limiting:

```

from slowapi import Limiter

Per endpoint limits
@limiter.limit("5/minute") # Login
@limiter.limit("100/hour") # API calls
@limiter.limit("1000/day") # General

IP-based limiting

```

```
Account-based limiting
Automatic blacklist
```

### 3: (Data Security)

#### 3.1 (Encryption at Rest) MongoDB Encryption:

```
encryption:
 keyManagement:
 provider: "local" # AWS KMS
 key: "256-bit AES key"

 encryptedCollections:
 - users ()
 - clinical_notes ()
 - analyses ()
 - otp_records (OTP codes)
```

#### Password Hashing:

```
import bcrypt

bcrypt cost factor
password_hash = bcrypt.hashpw(
 password.encode('utf-8'),
 bcrypt.gensalt(rounds=12) # 12 iterations
)

rainbow table attacks
brute force ()
```

#### Sensitive Data Encryption:

```
from cryptography.fernet import Fernet

AES-256
key = Fernet.generate_key()
cipher = Fernet(key)

#
encrypted = cipher.encrypt(sensitive_data.encode())

#
decrypted = cipher.decrypt(encrypted).decode()
```

#### 3.2 (PHI Protection) PHI (Protected Health Information):

```
:
- (Patient ID, Name, Age)
-
-
-

:
 AES-256
 Access Control
 Audit Logging
 Data Masking Logs
 Backup
```

### 3.3 (Backup & Recovery)

```
backup_strategy:
 frequency:
 full: " 2:00 AM"
 incremental: " 6 "

 retention:
 daily: "30 "
 weekly: "12 "
 monthly: "12 "

 encryption:
 algorithm: "AES-256-GCM"
 key_rotation: "90 "

 storage:
 primary: "MongoDB Atlas"
 secondary: "AWS S3 (encrypted)"

 testing:
 frequency: ""
 rto: "< 4 " # Recovery Time Objective
 rpo: "< 6 " # Recovery Point Objective
```

## 4: (Monitoring & Auditing)

### 4.1 (Audit Logs)

```
action
audit_log = {
 "user_id": "uuid",
 "user_email": "email",
 "action": "login|create_note|delete_user|...",
 "resource": "users|notes|analyses|...",
 "resource_id": "uuid",
 "details": {
 "changes": " ",
 "reason": " "
 },
 "ip_address": "xxx.xxx.xxx.xxx",
 "user_agent": "browser info",
 "timestamp": "ISO datetime",
 "status": "success|failure",
 "error": "error message if failed"
}

: 7 (HIPAA requirement)
```

### 4.2 (Real-time Monitoring)

```
Prometheus Metrics
metrics = {
 "failed_login_attempts": Counter,
 "api_errors": Counter,
 "response_time": Histogram,
 "active_users": Gauge,
 "database_queries": Counter
```

```

}

Alerts
alerts = [
 "5+ failed logins IP",
 "10+ API errors ",
 "Response time > 5 seconds",
 "Database connection failure",
 "Disk space < 10%"
]

```

### 4.3 (Intrusion Detection)

```

ids_rules:
- rule: "SQL injection attempt"
 pattern: "SELECT|UNION|DROP|INSERT"
 action: "block + alert"

- rule: "XSS attempt"
 pattern: "<script|javascript:|onerror="
 action: "sanitize + alert"

- rule: "Brute force attack"
 condition: "10+ failed logins in 5 min"
 action: "block IP + alert"

- rule: "Unusual activity"
 condition: "login from new country"
 action: "require MFA + alert"

```

---

```

1.
:
: 8

(!@#$$%^&*)

:

()
: 90 ()
 Email/OTP

:
bcrypt hashing

Salt

```

2.

```
lockout_policy = {
 "failed_attempts_threshold": 5,
 "lockout_duration": "30 minutes",
 "reset_after": "24 hours of no attempts",
 "notification": "Email to user + admin",
 "override": "Admin can unlock manually"
}
```

### 3. (Session Policy)

```
session_policy = {
 "duration": "24 hours",
 "idle_timeout": "2 hours",
 "secure": true, // HTTPS only
 "httpOnly": true, // No JavaScript access
 "sameSite": "Strict", // CSRF protection
 "refresh": "30 minutes before expiry",
 "concurrent": "3 max per user"
}
```

---

#### Phase 1: (Detection)

- 24/7
- Alert
- 

#### Phase 2: (Containment)

- 
- IP
- 
- 

#### Phase 3: (Investigation)

- Logs
- 
- 
- 

#### Phase 4: (Recovery)

- 
- 
- 
- 

#### Phase 5: (Lessons Learned)

- 
- 
-

-

---

## HIPAA Compliance Checklist

### Access Control ( )

- Unique user IDs
- Emergency access procedure
- Automatic logoff
- Encryption

### Audit Controls ( )

- Audit logs      PHI
- 6+
- 

### Integrity Controls ( )

- 
- Checksums
- Digital signatures

### Transmission Security ( )

- End-to-end encryption
- Integrity controls
- Network security

### Business Associate Agreement

- Third-parties
  -
- 

## Penetration Testing

frequency: " "

### tests:

- Vulnerability scanning
- Port scanning
- SQL injection
- XSS attacks
- CSRF attacks
- Authentication bypass
- Authorization bypass
- Session hijacking
- API security
- Mobile security (future)

### tools:

- OWASP ZAP
- Burp Suite
- Nmap
- Metasploit
- SQLMap

last\_test: "2025-11-XX"  
next\_test: "2026-02-XX"  
issues\_found: 0 Critical, 2 Medium, 5 Low

---

\*\* :\*\* - Email: security@cdi-system.sa - Phone: +966 XX XXX XXXX (24/7) - Emergency: +966 XX XXX XXXX

\*\* :\*\* - Email: security@cdi-system.sa - PGP Key: [attached]

\*\* (Bug Bounty):\*\* - Critical: 50,000 - High: 25,000 - Medium: 10,000 - Low: 5,000

---

\*\* 2025 CDI System - Cybersecurity Documentation - \*\*